In [21]:

```python
import numpy as np
import h5py
import matplotlib.pyplot as plt



def load_hdf5_data(train_path, test_path):
    with h5py.File(train_path, 'r') as f:
        print("Train file keys:", list(f.keys()))
        X_all = f['xdata'][:]
        y_all = f['ydata'][:]

    with h5py.File(test_path, 'r') as f:
        print("Test file keys:", list(f.keys()))
        X_test = f['xdata'][:]
        y_test = f['ydata'][:]

    print("Raw X_all shape:", X_all.shape)
    print("Raw y_all shape:", y_all.shape)
    print("Raw X_test shape:", X_test.shape)
    print("Raw y_test shape:", y_test.shape)

    return X_all, y_all, X_test, y_test


def preprocess_x(X):
    X = np.array(X, dtype=np.float32)

    if X.ndim == 3:
        X = X.reshape(X.shape[0], -1)
    elif X.ndim == 2:
        pass
    else:
        raise ValueError("Unexpected X shape: {}".format(X.shape))

    if X.max() > 1.0:
        X = X / 255.0

    return X


def preprocess_y(y):
    y = np.array(y)


    if y.ndim == 2 and y.shape[1] == 10:
        y = np.argmax(y, axis=1)
    else:
        y = y.astype(np.int64).reshape(-1)

    return y.astype(np.int64)


def train_val_split(X, y, train_size=50000):

    X = X[:60000]
    y = y[:60000]
```

```
    X_train = X[:train_size]
    y_train = y[:train_size]
    X_val = X[train_size:60000]
    y_val = y[train_size:60000]

    return X_train, y_train, X_val, y_val


def one_hot(y, num_classes=10):
    out = np.zeros((y.shape[0], num_classes), dtype=np.float32)
    out[np.arange(y.shape[0]), y] = 1.0
    return out



def relu(x):
    return np.maximum(0, x)


def relu_derivative(x):
    grad = np.zeros_like(x)
    grad[x > 0] = 1.0
    grad[x <= 0] = 0.0
    return grad


def tanh(x):
    return np.tanh(x)


def tanh_derivative(x):
    t = np.tanh(x)
    return 1.0 - t * t


def get_activation(name):
    if name == 'relu':
        return relu, relu_derivative
    elif name == 'tanh':
        return tanh, tanh_derivative
    else:
        raise ValueError("Unsupported activation: {}".format(name))



def softmax(logits):
    shifted = logits - np.max(logits, axis=1, keepdims=True)
    exp_scores = np.exp(shifted)
    probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)
    return probs


def cross_entropy_loss(probs, y_true_onehot, eps=1e-12):
    probs = np.clip(probs, eps, 1.0)
    loss = -np.sum(y_true_onehot * np.log(probs)) / y_true_onehot.shape[0]
    return loss


def compute_accuracy(probs, y_true):
    preds = np.argmax(probs, axis=1)
```

```
    return np.mean(preds == y_true)


def xavier_init(in_dim, out_dim):
    limit = np.sqrt(6.0 / (in_dim + out_dim))
    return np.random.uniform(-limit, limit, size=(in_dim, out_dim)).astype(np.fl


def he_init(in_dim, out_dim):
    std = np.sqrt(2.0 / in_dim)
    return (np.random.randn(in_dim, out_dim) * std).astype(np.float32)



class MLP:
    def __init__(self, layer_sizes, activation='relu', weight_decay=0.0, seed=42
        """
        例如:
        layer_sizes = [784, 256, 128, 10]
        """
        np.random.seed(seed)

        self.layer_sizes = layer_sizes
        self.num_layers = len(layer_sizes) - 1
        self.activation_name = activation
        self.act, self.act_deriv = get_activation(activation)
        self.weight_decay = weight_decay

        self.params = {}

        for i in range(1, len(layer_sizes)):
            in_dim = layer_sizes[i - 1]
            out_dim = layer_sizes[i]

            if i < len(layer_sizes) - 1:
                if activation == 'relu':
                    W = he_init(in_dim, out_dim)
                else:
                    W = xavier_init(in_dim, out_dim)
            else:
                W = xavier_init(in_dim, out_dim)

            b = np.zeros((1, out_dim), dtype=np.float32)

            self.params['W' + str(i)] = W
            self.params['b' + str(i)] = b

    def forward(self, X):
        cache = {}
        cache['A0'] = X
        A = X


        for i in range(1, self.num_layers):
            W = self.params['W' + str(i)]
            b = self.params['b' + str(i)]

            Z = A @ W + b
            A = self.act(Z)
```

```
            cache['Z' + str(i)] = Z
            cache['A' + str(i)] = A


        W = self.params['W' + str(self.num_layers)]
        b = self.params['b' + str(self.num_layers)]

        ZL = A @ W + b
        probs = softmax(ZL)

        cache['Z' + str(self.num_layers)] = ZL
        cache['A' + str(self.num_layers)] = probs

        return cache, probs

    def backward(self, cache, y_onehot):
        grads = {}
        m = y_onehot.shape[0]
        L = self.num_layers

        probs = cache['A' + str(L)]
        dZ = (probs - y_onehot) / m


        A_prev = cache['A' + str(L - 1)] if L - 1 >= 1 else cache['A0']
        grads['dW' + str(L)] = A_prev.T @ dZ + self.weight_decay * self.params['
        grads['db' + str(L)] = np.sum(dZ, axis=0, keepdims=True)

        dA_prev = dZ @ self.params['W' + str(L)].T


        for i in range(L - 1, 0, -1):
            Z = cache['Z' + str(i)]
            dZ = dA_prev * self.act_deriv(Z)

            A_prev = cache['A' + str(i - 1)] if i - 1 >= 1 else cache['A0']

            grads['dW' + str(i)] = A_prev.T @ dZ + self.weight_decay * self.para
            grads['db' + str(i)] = np.sum(dZ, axis=0, keepdims=True)

            if i > 1:
                dA_prev = dZ @ self.params['W' + str(i)].T

        return grads

    def update_params(self, grads, lr):
        for i in range(1, self.num_layers + 1):
            self.params['W' + str(i)] -= lr * grads['dW' + str(i)]
            self.params['b' + str(i)] -= lr * grads['db' + str(i)]

    def predict_proba(self, X):
        _, probs = self.forward(X)
        return probs

    def predict(self, X):
        probs = self.predict_proba(X)
        return np.argmax(probs, axis=1)

    def evaluate(self, X, y):
        _, probs = self.forward(X)
```

```
        y_onehot = one_hot(y, num_classes=10)

        loss = cross_entropy_loss(probs, y_onehot)
        acc = compute_accuracy(probs, y)

        return loss, acc



def get_minibatches(X, y, batch_size, shuffle=True):
    n = X.shape[0]
    indices = np.arange(n)

    if shuffle:
        np.random.shuffle(indices)

    for start in range(0, n, batch_size):
        end = start + batch_size
        batch_idx = indices[start:end]
        yield X[batch_idx], y[batch_idx]


def train_model(model, X_train, y_train, X_val, y_val,
                epochs=50, batch_size=100, initial_lr=0.01,
                decay_epochs=(20, 40), verbose=True):

    history = {
        'train_loss': [],
        'train_acc': [],
        'val_loss': [],
        'val_acc': [],
        'lr': []
    }

    lr = initial_lr

    for epoch in range(epochs):
        if epoch in decay_epochs:
            lr = lr / 2.0


        for Xb, yb in get_minibatches(X_train, y_train, batch_size=batch_size, s
            yb_onehot = one_hot(yb, num_classes=10)

            cache, probs = model.forward(Xb)
            grads = model.backward(cache, yb_onehot)
            model.update_params(grads, lr)

        train_loss, train_acc = model.evaluate(X_train, y_train)
        val_loss, val_acc = model.evaluate(X_val, y_val)

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)
        history['lr'].append(lr)

        if verbose:
            print("Epoch {:02d} | lr={:.5f} | train_loss={:.4f}, train_acc={:.4f
                epoch + 1, lr, train_loss, train_acc, val_loss, val_acc
```

```
            ))

    return history



def plot_histories(results, decay_epochs=(20, 40)):
    for name, hist in results.items():
        plt.figure(figsize=(8, 5))

        epochs = np.arange(1, len(hist['train_acc']) + 1)

        plt.plot(epochs, hist['train_acc'], label='Training Accuracy')
        plt.plot(epochs, hist['val_acc'], label='Validation Accuracy')

        for e in decay_epochs:
            plt.axvline(x=e, linestyle='--', label='LR Decay' if e == decay_epoc

        plt.xlabel('Epoch Number')
        plt.ylabel('Accuracy')
        plt.title('Configuration: ' + name)
        plt.legend()
        plt.grid(True)
        plt.show()



def run_experiments(train_path, test_path):
    X_all, y_all, X_test, y_test = load_hdf5_data(train_path, test_path)

    X_all = preprocess_x(X_all)
    y_all = preprocess_y(y_all)
    X_test = preprocess_x(X_test)
    y_test = preprocess_y(y_test)

    X_train, y_train, X_val, y_val = train_val_split(X_all, y_all, train_size=50

    print("Processed X_all:", X_all.shape)
    print("Processed y_all:", y_all.shape)
    print("X_train:", X_train.shape)
    print("y_train:", y_train.shape)
    print("X_val:", X_val.shape)
    print("y_val:", y_val.shape)
    print("X_test:", X_test.shape)
    print("y_test:", y_test.shape)

    print("Unique labels in y_train:", np.unique(y_train))
    print("Unique labels in y_val:", np.unique(y_val))
    print("Unique labels in y_test:", np.unique(y_test))

    layer_sizes = [784, 256, 128, 10]
    batch_size = 100
    epochs = 50
    decay_epochs = (20, 40)
    weight_decay = 1e-4

    activations = ['relu', 'tanh']
    learning_rates = [0.1, 0.01, 0.001]

    all_results = {}
```

```python
best_config = None
best_val_acc = -1.0

for act_name in activations:
    for lr in learning_rates:
        config_name = act_name + "_lr" + str(lr)

        print("\n" + "=" * 70)
        print("Training config:", config_name)
        print("=" * 70)

        model = MLP(
            layer_sizes=layer_sizes,
            activation=act_name,
            weight_decay=weight_decay,
            seed=42
        )

        history = train_model(
            model=model,
            X_train=X_train,
            y_train=y_train,
            X_val=X_val,
            y_val=y_val,
            epochs=epochs,
            batch_size=batch_size,
            initial_lr=lr,
            decay_epochs=decay_epochs,
            verbose=True
        )

        all_results[config_name] = history

        max_val_acc = max(history['val_acc'])
        print("Best val acc for {}: {:.4f}".format(config_name, max_val_acc)

        if max_val_acc > best_val_acc:
            best_val_acc = max_val_acc
            best_config = {
                'activation': act_name,
                'lr': lr,
                'name': config_name
            }

print("\nBest configuration:")
print(best_config)
print("Best validation accuracy:", best_val_acc)

plot_histories(all_results, decay_epochs=decay_epochs)


print("\n" + "=" * 70)
print("Retraining on all 60,000 training images...")
print("=" * 70)

X_full_train = X_all[:60000]
y_full_train = y_all[:60000]

best_model = MLP(
    layer_sizes=layer_sizes,
```

```
        activation=best_config['activation'],
        weight_decay=weight_decay,
        seed=42
    )

    lr = best_config['lr']
    final_epochs = 50

    for epoch in range(final_epochs):
        if epoch in decay_epochs:
            lr = lr / 2.0

        for Xb, yb in get_minibatches(X_full_train, y_full_train, batch_size=bat
            yb_onehot = one_hot(yb, num_classes=10)

            cache, probs = best_model.forward(Xb)
            grads = best_model.backward(cache, yb_onehot)
            best_model.update_params(grads, lr)

        train_loss, train_acc = best_model.evaluate(X_full_train, y_full_train)
        print("[Full Train] Epoch {:02d} | lr={:.5f} | loss={:.4f}, acc={:.4f}".
            epoch + 1, lr, train_loss, train_acc
        ))

    test_loss, test_acc = best_model.evaluate(X_test, y_test)
    print("\nFinal Test Accuracy:", test_acc)
    print("Final Test Loss:", test_loss)

    return {
        'all_results': all_results,
        'best_config': best_config,
        'best_val_acc': best_val_acc,
        'final_test_acc': test_acc,
        'final_test_loss': test_loss
    }



if __name__ == '__main__':
    train_path = 'mnist_traindata.hdf5'
    test_path = 'mnist_testdata.hdf5'

    results = run_experiments(train_path, test_path)

    print("\n===== Summary =====")
    print("Best config:", results['best_config'])
    print("Best val acc:", results['best_val_acc'])
    print("Final test acc:", results['final_test_acc'])
    print("Final test loss:", results['final_test_loss'])
```

```
Train file keys: ['xdata', 'ydata']
Test file keys: ['xdata', 'ydata']
Raw X_all shape: (60000, 784)
Raw y_all shape: (60000, 10)
Raw X_test shape: (10000, 784)
Raw y_test shape: (10000, 10)
Processed X_all: (60000, 784)
Processed y_all: (60000,)
X_train: (50000, 784)
y_train: (50000,)
X_val: (10000, 784)
y_val: (10000,)
X_test: (10000, 784)
y_test: (10000,)
Unique labels in y_train: [0 1 2 3 4 5 6 7 8 9]
Unique labels in y_val: [0 1 2 3 4 5 6 7 8 9]
Unique labels in y_test: [0 1 2 3 4 5 6 7 8 9]

======================================================================
Training config: relu_lr0.1
======================================================================
Epoch 01 | lr=0.10000 | train_loss=0.2151, train_acc=0.9386 | val_loss=0.2049, va
l_acc=0.9427
Epoch 02 | lr=0.10000 | train_loss=0.1479, train_acc=0.9578 | val_loss=0.1478, va
l_acc=0.9590
Epoch 03 | lr=0.10000 | train_loss=0.1117, train_acc=0.9687 | val_loss=0.1249, va
l_acc=0.9657
Epoch 04 | lr=0.10000 | train_loss=0.0914, train_acc=0.9741 | val_loss=0.1120, va
l_acc=0.9670
Epoch 05 | lr=0.10000 | train_loss=0.0735, train_acc=0.9795 | val_loss=0.0973, va
l_acc=0.9707
Epoch 06 | lr=0.10000 | train_loss=0.0708, train_acc=0.9791 | val_loss=0.0945, va
l_acc=0.9730
Epoch 07 | lr=0.10000 | train_loss=0.0578, train_acc=0.9838 | val_loss=0.0957, va
l_acc=0.9721
Epoch 08 | lr=0.10000 | train_loss=0.0455, train_acc=0.9878 | val_loss=0.0840, va
l_acc=0.9751
Epoch 09 | lr=0.10000 | train_loss=0.0392, train_acc=0.9897 | val_loss=0.0818, va
l_acc=0.9748
Epoch 10 | lr=0.10000 | train_loss=0.0311, train_acc=0.9927 | val_loss=0.0780, va
l_acc=0.9766
Epoch 11 | lr=0.10000 | train_loss=0.0280, train_acc=0.9940 | val_loss=0.0772, va
l_acc=0.9765
Epoch 12 | lr=0.10000 | train_loss=0.0349, train_acc=0.9902 | val_loss=0.0858, va
l_acc=0.9747
Epoch 13 | lr=0.10000 | train_loss=0.0227, train_acc=0.9957 | val_loss=0.0749, va
l_acc=0.9769
Epoch 14 | lr=0.10000 | train_loss=0.0202, train_acc=0.9964 | val_loss=0.0776, va
l_acc=0.9780
Epoch 15 | lr=0.10000 | train_loss=0.0216, train_acc=0.9956 | val_loss=0.0795, va
l_acc=0.9767
Epoch 16 | lr=0.10000 | train_loss=0.0155, train_acc=0.9979 | val_loss=0.0758, va
l_acc=0.9776
Epoch 17 | lr=0.10000 | train_loss=0.0128, train_acc=0.9986 | val_loss=0.0731, va
l_acc=0.9787
Epoch 18 | lr=0.10000 | train_loss=0.0120, train_acc=0.9984 | val_loss=0.0714, va
l_acc=0.9798
Epoch 19 | lr=0.10000 | train_loss=0.0111, train_acc=0.9986 | val_loss=0.0755, va
l_acc=0.9767
Epoch 20 | lr=0.10000 | train_loss=0.0090, train_acc=0.9993 | val_loss=0.0710, va
```

```
l_acc=0.9796
Epoch 21 | lr=0.05000 | train_loss=0.0076, train_acc=0.9995 | val_loss=0.0707, va
l_acc=0.9790
Epoch 22 | lr=0.05000 | train_loss=0.0073, train_acc=0.9996 | val_loss=0.0697, va
l_acc=0.9787
Epoch 23 | lr=0.05000 | train_loss=0.0074, train_acc=0.9994 | val_loss=0.0715, va
l_acc=0.9790
Epoch 24 | lr=0.05000 | train_loss=0.0068, train_acc=0.9996 | val_loss=0.0703, va
l_acc=0.9797
Epoch 25 | lr=0.05000 | train_loss=0.0065, train_acc=0.9996 | val_loss=0.0709, va
l_acc=0.9794
Epoch 26 | lr=0.05000 | train_loss=0.0062, train_acc=0.9997 | val_loss=0.0714, va
l_acc=0.9793
Epoch 27 | lr=0.05000 | train_loss=0.0061, train_acc=0.9997 | val_loss=0.0709, va
l_acc=0.9788
Epoch 28 | lr=0.05000 | train_loss=0.0059, train_acc=0.9998 | val_loss=0.0711, va
l_acc=0.9795
Epoch 29 | lr=0.05000 | train_loss=0.0057, train_acc=0.9998 | val_loss=0.0713, va
l_acc=0.9801
Epoch 30 | lr=0.05000 | train_loss=0.0055, train_acc=0.9998 | val_loss=0.0717, va
l_acc=0.9797
Epoch 31 | lr=0.05000 | train_loss=0.0053, train_acc=0.9998 | val_loss=0.0715, va
l_acc=0.9792
Epoch 32 | lr=0.05000 | train_loss=0.0052, train_acc=0.9999 | val_loss=0.0722, va
l_acc=0.9794
Epoch 33 | lr=0.05000 | train_loss=0.0051, train_acc=0.9999 | val_loss=0.0723, va
l_acc=0.9793
Epoch 34 | lr=0.05000 | train_loss=0.0050, train_acc=0.9999 | val_loss=0.0717, va
l_acc=0.9797
Epoch 35 | lr=0.05000 | train_loss=0.0047, train_acc=0.9999 | val_loss=0.0715, va
l_acc=0.9797
Epoch 36 | lr=0.05000 | train_loss=0.0047, train_acc=0.9999 | val_loss=0.0712, va
l_acc=0.9799
Epoch 37 | lr=0.05000 | train_loss=0.0047, train_acc=0.9999 | val_loss=0.0728, va
l_acc=0.9799
Epoch 38 | lr=0.05000 | train_loss=0.0044, train_acc=0.9999 | val_loss=0.0716, va
l_acc=0.9799
Epoch 39 | lr=0.05000 | train_loss=0.0043, train_acc=0.9999 | val_loss=0.0718, va
l_acc=0.9797
Epoch 40 | lr=0.05000 | train_loss=0.0042, train_acc=0.9999 | val_loss=0.0717, va
l_acc=0.9802
Epoch 41 | lr=0.02500 | train_loss=0.0041, train_acc=1.0000 | val_loss=0.0719, va
l_acc=0.9799
Epoch 42 | lr=0.02500 | train_loss=0.0041, train_acc=1.0000 | val_loss=0.0720, va
l_acc=0.9800
Epoch 43 | lr=0.02500 | train_loss=0.0040, train_acc=1.0000 | val_loss=0.0718, va
l_acc=0.9798
Epoch 44 | lr=0.02500 | train_loss=0.0040, train_acc=1.0000 | val_loss=0.0720, va
l_acc=0.9796
Epoch 45 | lr=0.02500 | train_loss=0.0040, train_acc=1.0000 | val_loss=0.0718, va
l_acc=0.9802
Epoch 46 | lr=0.02500 | train_loss=0.0039, train_acc=1.0000 | val_loss=0.0719, va
l_acc=0.9800
Epoch 47 | lr=0.02500 | train_loss=0.0039, train_acc=1.0000 | val_loss=0.0721, va
l_acc=0.9798
Epoch 48 | lr=0.02500 | train_loss=0.0039, train_acc=1.0000 | val_loss=0.0719, va
l_acc=0.9798
Epoch 49 | lr=0.02500 | train_loss=0.0039, train_acc=1.0000 | val_loss=0.0719, va
l_acc=0.9797
Epoch 50 | lr=0.02500 | train_loss=0.0038, train_acc=1.0000 | val_loss=0.0719, va
```

```
l_acc=0.9801
Best val acc for relu_lr0.1: 0.9802

======================================================================
Training config: relu_lr0.01
======================================================================
Epoch 01 | lr=0.01000 | train_loss=0.5193, train_acc=0.8648 | val_loss=0.4776, va
l_acc=0.8789
Epoch 02 | lr=0.01000 | train_loss=0.3785, train_acc=0.8950 | val_loss=0.3478, va
l_acc=0.9020
Epoch 03 | lr=0.01000 | train_loss=0.3256, train_acc=0.9088 | val_loss=0.3024, va
l_acc=0.9146
Epoch 04 | lr=0.01000 | train_loss=0.2962, train_acc=0.9152 | val_loss=0.2763, va
l_acc=0.9217
Epoch 05 | lr=0.01000 | train_loss=0.2729, train_acc=0.9233 | val_loss=0.2558, va
l_acc=0.9291
Epoch 06 | lr=0.01000 | train_loss=0.2541, train_acc=0.9293 | val_loss=0.2392, va
l_acc=0.9323
Epoch 07 | lr=0.01000 | train_loss=0.2392, train_acc=0.9323 | val_loss=0.2281, va
l_acc=0.9356
Epoch 08 | lr=0.01000 | train_loss=0.2254, train_acc=0.9366 | val_loss=0.2164, va
l_acc=0.9401
Epoch 09 | lr=0.01000 | train_loss=0.2148, train_acc=0.9399 | val_loss=0.2065, va
l_acc=0.9429
Epoch 10 | lr=0.01000 | train_loss=0.2037, train_acc=0.9434 | val_loss=0.1974, va
l_acc=0.9461
Epoch 11 | lr=0.01000 | train_loss=0.1946, train_acc=0.9461 | val_loss=0.1895, va
l_acc=0.9482
Epoch 12 | lr=0.01000 | train_loss=0.1884, train_acc=0.9468 | val_loss=0.1845, va
l_acc=0.9504
Epoch 13 | lr=0.01000 | train_loss=0.1785, train_acc=0.9497 | val_loss=0.1757, va
l_acc=0.9532
Epoch 14 | lr=0.01000 | train_loss=0.1715, train_acc=0.9519 | val_loss=0.1711, va
l_acc=0.9544
Epoch 15 | lr=0.01000 | train_loss=0.1662, train_acc=0.9530 | val_loss=0.1653, va
l_acc=0.9549
Epoch 16 | lr=0.01000 | train_loss=0.1603, train_acc=0.9549 | val_loss=0.1618, va
l_acc=0.9550
Epoch 17 | lr=0.01000 | train_loss=0.1560, train_acc=0.9570 | val_loss=0.1590, va
l_acc=0.9568
Epoch 18 | lr=0.01000 | train_loss=0.1492, train_acc=0.9583 | val_loss=0.1533, va
l_acc=0.9565
Epoch 19 | lr=0.01000 | train_loss=0.1429, train_acc=0.9604 | val_loss=0.1474, va
l_acc=0.9595
Epoch 20 | lr=0.01000 | train_loss=0.1384, train_acc=0.9618 | val_loss=0.1444, va
l_acc=0.9601
Epoch 21 | lr=0.00500 | train_loss=0.1355, train_acc=0.9627 | val_loss=0.1417, va
l_acc=0.9605
Epoch 22 | lr=0.00500 | train_loss=0.1334, train_acc=0.9634 | val_loss=0.1399, va
l_acc=0.9614
Epoch 23 | lr=0.00500 | train_loss=0.1312, train_acc=0.9639 | val_loss=0.1388, va
l_acc=0.9612
Epoch 24 | lr=0.00500 | train_loss=0.1298, train_acc=0.9642 | val_loss=0.1381, va
l_acc=0.9627
Epoch 25 | lr=0.00500 | train_loss=0.1277, train_acc=0.9647 | val_loss=0.1367, va
l_acc=0.9623
Epoch 26 | lr=0.00500 | train_loss=0.1252, train_acc=0.9657 | val_loss=0.1345, va
l_acc=0.9626
Epoch 27 | lr=0.00500 | train_loss=0.1237, train_acc=0.9663 | val_loss=0.1331, va
l_acc=0.9623
```

```
Epoch 28 | lr=0.00500 | train_loss=0.1214, train_acc=0.9668 | val_loss=0.1316, va
l_acc=0.9637
Epoch 29 | lr=0.00500 | train_loss=0.1196, train_acc=0.9673 | val_loss=0.1303, va
l_acc=0.9636
Epoch 30 | lr=0.00500 | train_loss=0.1178, train_acc=0.9678 | val_loss=0.1292, va
l_acc=0.9634
Epoch 31 | lr=0.00500 | train_loss=0.1164, train_acc=0.9684 | val_loss=0.1281, va
l_acc=0.9635
Epoch 32 | lr=0.00500 | train_loss=0.1147, train_acc=0.9685 | val_loss=0.1275, va
l_acc=0.9648
Epoch 33 | lr=0.00500 | train_loss=0.1130, train_acc=0.9690 | val_loss=0.1264, va
l_acc=0.9645
Epoch 34 | lr=0.00500 | train_loss=0.1114, train_acc=0.9693 | val_loss=0.1248, va
l_acc=0.9650
Epoch 35 | lr=0.00500 | train_loss=0.1098, train_acc=0.9701 | val_loss=0.1243, va
l_acc=0.9657
Epoch 36 | lr=0.00500 | train_loss=0.1084, train_acc=0.9701 | val_loss=0.1230, va
l_acc=0.9660
Epoch 37 | lr=0.00500 | train_loss=0.1081, train_acc=0.9700 | val_loss=0.1234, va
l_acc=0.9647
Epoch 38 | lr=0.00500 | train_loss=0.1055, train_acc=0.9712 | val_loss=0.1207, va
l_acc=0.9662
Epoch 39 | lr=0.00500 | train_loss=0.1040, train_acc=0.9716 | val_loss=0.1200, va
l_acc=0.9663
Epoch 40 | lr=0.00500 | train_loss=0.1031, train_acc=0.9717 | val_loss=0.1197, va
l_acc=0.9665
Epoch 41 | lr=0.00250 | train_loss=0.1016, train_acc=0.9719 | val_loss=0.1185, va
l_acc=0.9665
Epoch 42 | lr=0.00250 | train_loss=0.1010, train_acc=0.9722 | val_loss=0.1180, va
l_acc=0.9665
Epoch 43 | lr=0.00250 | train_loss=0.1005, train_acc=0.9725 | val_loss=0.1175, va
l_acc=0.9664
Epoch 44 | lr=0.00250 | train_loss=0.0998, train_acc=0.9727 | val_loss=0.1172, va
l_acc=0.9670
Epoch 45 | lr=0.00250 | train_loss=0.0991, train_acc=0.9729 | val_loss=0.1164, va
l_acc=0.9667
Epoch 46 | lr=0.00250 | train_loss=0.0984, train_acc=0.9730 | val_loss=0.1164, va
l_acc=0.9670
Epoch 47 | lr=0.00250 | train_loss=0.0978, train_acc=0.9733 | val_loss=0.1162, va
l_acc=0.9673
Epoch 48 | lr=0.00250 | train_loss=0.0972, train_acc=0.9734 | val_loss=0.1155, va
l_acc=0.9674
Epoch 49 | lr=0.00250 | train_loss=0.0969, train_acc=0.9735 | val_loss=0.1150, va
l_acc=0.9671
Epoch 50 | lr=0.00250 | train_loss=0.0960, train_acc=0.9736 | val_loss=0.1146, va
l_acc=0.9673
Best val acc for relu_lr0.01: 0.9674

====================================================================
Training config: relu_lr0.001
====================================================================
Epoch 01 | lr=0.00100 | train_loss=1.8939, train_acc=0.5303 | val_loss=1.8756, va
l_acc=0.5546
Epoch 02 | lr=0.00100 | train_loss=1.4974, train_acc=0.6996 | val_loss=1.4692, va
l_acc=0.7244
Epoch 03 | lr=0.00100 | train_loss=1.1771, train_acc=0.7575 | val_loss=1.1400, va
l_acc=0.7862
Epoch 04 | lr=0.00100 | train_loss=0.9571, train_acc=0.7937 | val_loss=0.9138, va
l_acc=0.8168
Epoch 05 | lr=0.00100 | train_loss=0.8109, train_acc=0.8174 | val_loss=0.7650, va
```

```
l_acc=0.8385
Epoch 06 | lr=0.00100 | train_loss=0.7113, train_acc=0.8325 | val_loss=0.6653, va
l_acc=0.8535
Epoch 07 | lr=0.00100 | train_loss=0.6406, train_acc=0.8443 | val_loss=0.5950, va
l_acc=0.8639
Epoch 08 | lr=0.00100 | train_loss=0.5884, train_acc=0.8547 | val_loss=0.5440, va
l_acc=0.8720
Epoch 09 | lr=0.00100 | train_loss=0.5487, train_acc=0.8617 | val_loss=0.5055, va
l_acc=0.8758
Epoch 10 | lr=0.00100 | train_loss=0.5171, train_acc=0.8678 | val_loss=0.4751, va
l_acc=0.8817
Epoch 11 | lr=0.00100 | train_loss=0.4917, train_acc=0.8725 | val_loss=0.4514, va
l_acc=0.8843
Epoch 12 | lr=0.00100 | train_loss=0.4705, train_acc=0.8770 | val_loss=0.4314, va
l_acc=0.8876
Epoch 13 | lr=0.00100 | train_loss=0.4526, train_acc=0.8803 | val_loss=0.4149, va
l_acc=0.8908
Epoch 14 | lr=0.00100 | train_loss=0.4373, train_acc=0.8837 | val_loss=0.4009, va
l_acc=0.8935
Epoch 15 | lr=0.00100 | train_loss=0.4240, train_acc=0.8860 | val_loss=0.3889, va
l_acc=0.8953
Epoch 16 | lr=0.00100 | train_loss=0.4122, train_acc=0.8888 | val_loss=0.3783, va
l_acc=0.8971
Epoch 17 | lr=0.00100 | train_loss=0.4019, train_acc=0.8914 | val_loss=0.3693, va
l_acc=0.8990
Epoch 18 | lr=0.00100 | train_loss=0.3925, train_acc=0.8928 | val_loss=0.3606, va
l_acc=0.9016
Epoch 19 | lr=0.00100 | train_loss=0.3840, train_acc=0.8946 | val_loss=0.3530, va
l_acc=0.9025
Epoch 20 | lr=0.00100 | train_loss=0.3763, train_acc=0.8964 | val_loss=0.3464, va
l_acc=0.9046
Epoch 21 | lr=0.00050 | train_loss=0.3728, train_acc=0.8970 | val_loss=0.3431, va
l_acc=0.9050
Epoch 22 | lr=0.00050 | train_loss=0.3693, train_acc=0.8976 | val_loss=0.3403, va
l_acc=0.9059
Epoch 23 | lr=0.00050 | train_loss=0.3660, train_acc=0.8981 | val_loss=0.3373, va
l_acc=0.9064
Epoch 24 | lr=0.00050 | train_loss=0.3628, train_acc=0.8989 | val_loss=0.3347, va
l_acc=0.9070
Epoch 25 | lr=0.00050 | train_loss=0.3598, train_acc=0.8999 | val_loss=0.3318, va
l_acc=0.9079
Epoch 26 | lr=0.00050 | train_loss=0.3568, train_acc=0.9007 | val_loss=0.3293, va
l_acc=0.9086
Epoch 27 | lr=0.00050 | train_loss=0.3540, train_acc=0.9015 | val_loss=0.3267, va
l_acc=0.9088
Epoch 28 | lr=0.00050 | train_loss=0.3513, train_acc=0.9020 | val_loss=0.3245, va
l_acc=0.9090
Epoch 29 | lr=0.00050 | train_loss=0.3487, train_acc=0.9029 | val_loss=0.3221, va
l_acc=0.9093
Epoch 30 | lr=0.00050 | train_loss=0.3461, train_acc=0.9033 | val_loss=0.3199, va
l_acc=0.9100
Epoch 31 | lr=0.00050 | train_loss=0.3436, train_acc=0.9040 | val_loss=0.3178, va
l_acc=0.9102
Epoch 32 | lr=0.00050 | train_loss=0.3412, train_acc=0.9044 | val_loss=0.3157, va
l_acc=0.9110
Epoch 33 | lr=0.00050 | train_loss=0.3388, train_acc=0.9052 | val_loss=0.3137, va
l_acc=0.9114
Epoch 34 | lr=0.00050 | train_loss=0.3366, train_acc=0.9058 | val_loss=0.3118, va
l_acc=0.9116
Epoch 35 | lr=0.00050 | train_loss=0.3344, train_acc=0.9065 | val_loss=0.3100, va
```

```
l_acc=0.9131
Epoch 36 | lr=0.00050 | train_loss=0.3323, train_acc=0.9067 | val_loss=0.3080, va
l_acc=0.9133
Epoch 37 | lr=0.00050 | train_loss=0.3302, train_acc=0.9076 | val_loss=0.3062, va
l_acc=0.9136
Epoch 38 | lr=0.00050 | train_loss=0.3281, train_acc=0.9081 | val_loss=0.3043, va
l_acc=0.9141
Epoch 39 | lr=0.00050 | train_loss=0.3261, train_acc=0.9086 | val_loss=0.3026, va
l_acc=0.9145
Epoch 40 | lr=0.00050 | train_loss=0.3242, train_acc=0.9091 | val_loss=0.3011, va
l_acc=0.9148
Epoch 41 | lr=0.00025 | train_loss=0.3232, train_acc=0.9094 | val_loss=0.3001, va
l_acc=0.9152
Epoch 42 | lr=0.00025 | train_loss=0.3223, train_acc=0.9097 | val_loss=0.2993, va
l_acc=0.9156
Epoch 43 | lr=0.00025 | train_loss=0.3213, train_acc=0.9099 | val_loss=0.2985, va
l_acc=0.9161
Epoch 44 | lr=0.00025 | train_loss=0.3204, train_acc=0.9101 | val_loss=0.2978, va
l_acc=0.9158
Epoch 45 | lr=0.00025 | train_loss=0.3195, train_acc=0.9103 | val_loss=0.2969, va
l_acc=0.9164
Epoch 46 | lr=0.00025 | train_loss=0.3186, train_acc=0.9105 | val_loss=0.2961, va
l_acc=0.9168
Epoch 47 | lr=0.00025 | train_loss=0.3177, train_acc=0.9106 | val_loss=0.2954, va
l_acc=0.9168
Epoch 48 | lr=0.00025 | train_loss=0.3168, train_acc=0.9107 | val_loss=0.2947, va
l_acc=0.9171
Epoch 49 | lr=0.00025 | train_loss=0.3160, train_acc=0.9110 | val_loss=0.2938, va
l_acc=0.9178
Epoch 50 | lr=0.00025 | train_loss=0.3151, train_acc=0.9111 | val_loss=0.2931, va
l_acc=0.9178
Best val acc for relu_lr0.001: 0.9178

====================================================================
Training config: tanh_lr0.1
====================================================================
Epoch 01 | lr=0.10000 | train_loss=0.2753, train_acc=0.9217 | val_loss=0.2608, va
l_acc=0.9263
Epoch 02 | lr=0.10000 | train_loss=0.2072, train_acc=0.9408 | val_loss=0.2030, va
l_acc=0.9432
Epoch 03 | lr=0.10000 | train_loss=0.1703, train_acc=0.9508 | val_loss=0.1725, va
l_acc=0.9523
Epoch 04 | lr=0.10000 | train_loss=0.1376, train_acc=0.9608 | val_loss=0.1438, va
l_acc=0.9610
Epoch 05 | lr=0.10000 | train_loss=0.1195, train_acc=0.9653 | val_loss=0.1321, va
l_acc=0.9642
Epoch 06 | lr=0.10000 | train_loss=0.1032, train_acc=0.9703 | val_loss=0.1194, va
l_acc=0.9653
Epoch 07 | lr=0.10000 | train_loss=0.0938, train_acc=0.9729 | val_loss=0.1118, va
l_acc=0.9677
Epoch 08 | lr=0.10000 | train_loss=0.0814, train_acc=0.9777 | val_loss=0.1050, va
l_acc=0.9698
Epoch 09 | lr=0.10000 | train_loss=0.0761, train_acc=0.9788 | val_loss=0.0996, va
l_acc=0.9719
Epoch 10 | lr=0.10000 | train_loss=0.0658, train_acc=0.9823 | val_loss=0.0941, va
l_acc=0.9749
Epoch 11 | lr=0.10000 | train_loss=0.0651, train_acc=0.9818 | val_loss=0.0967, va
l_acc=0.9726
Epoch 12 | lr=0.10000 | train_loss=0.0542, train_acc=0.9856 | val_loss=0.0874, va
l_acc=0.9759
```

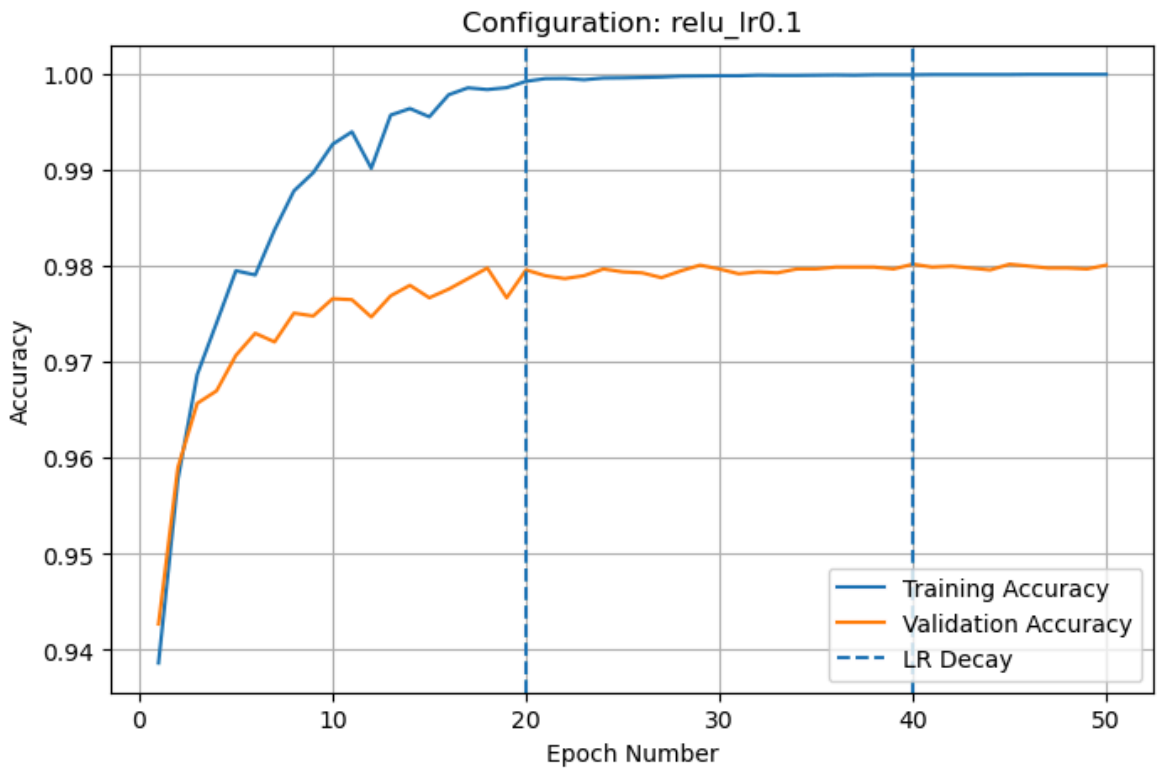
```
Epoch 13 | lr=0.10000 | train_loss=0.0510, train_acc=0.9865 | val_loss=0.0857, va
l_acc=0.9744
Epoch 14 | lr=0.10000 | train_loss=0.0469, train_acc=0.9881 | val_loss=0.0851, va
l_acc=0.9745
Epoch 15 | lr=0.10000 | train_loss=0.0427, train_acc=0.9889 | val_loss=0.0826, va
l_acc=0.9748
Epoch 16 | lr=0.10000 | train_loss=0.0387, train_acc=0.9905 | val_loss=0.0801, va
l_acc=0.9766
Epoch 17 | lr=0.10000 | train_loss=0.0349, train_acc=0.9920 | val_loss=0.0777, va
l_acc=0.9767
Epoch 18 | lr=0.10000 | train_loss=0.0327, train_acc=0.9929 | val_loss=0.0769, va
l_acc=0.9772
Epoch 19 | lr=0.10000 | train_loss=0.0302, train_acc=0.9933 | val_loss=0.0759, va
l_acc=0.9772
Epoch 20 | lr=0.10000 | train_loss=0.0284, train_acc=0.9938 | val_loss=0.0746, va
l_acc=0.9772
Epoch 21 | lr=0.05000 | train_loss=0.0256, train_acc=0.9952 | val_loss=0.0731, va
l_acc=0.9774
Epoch 22 | lr=0.05000 | train_loss=0.0248, train_acc=0.9956 | val_loss=0.0730, va
l_acc=0.9783
Epoch 23 | lr=0.05000 | train_loss=0.0235, train_acc=0.9961 | val_loss=0.0723, va
l_acc=0.9776
Epoch 24 | lr=0.05000 | train_loss=0.0225, train_acc=0.9965 | val_loss=0.0723, va
l_acc=0.9784
Epoch 25 | lr=0.05000 | train_loss=0.0224, train_acc=0.9964 | val_loss=0.0723, va
l_acc=0.9785
Epoch 26 | lr=0.05000 | train_loss=0.0209, train_acc=0.9970 | val_loss=0.0712, va
l_acc=0.9791
Epoch 27 | lr=0.05000 | train_loss=0.0216, train_acc=0.9966 | val_loss=0.0722, va
l_acc=0.9779
Epoch 28 | lr=0.05000 | train_loss=0.0200, train_acc=0.9970 | val_loss=0.0721, va
l_acc=0.9771
Epoch 29 | lr=0.05000 | train_loss=0.0194, train_acc=0.9972 | val_loss=0.0709, va
l_acc=0.9791
Epoch 30 | lr=0.05000 | train_loss=0.0183, train_acc=0.9978 | val_loss=0.0711, va
l_acc=0.9787
Epoch 31 | lr=0.05000 | train_loss=0.0179, train_acc=0.9977 | val_loss=0.0707, va
l_acc=0.9780
Epoch 32 | lr=0.05000 | train_loss=0.0175, train_acc=0.9978 | val_loss=0.0710, va
l_acc=0.9780
Epoch 33 | lr=0.05000 | train_loss=0.0167, train_acc=0.9981 | val_loss=0.0708, va
l_acc=0.9777
Epoch 34 | lr=0.05000 | train_loss=0.0173, train_acc=0.9979 | val_loss=0.0710, va
l_acc=0.9787
Epoch 35 | lr=0.05000 | train_loss=0.0155, train_acc=0.9982 | val_loss=0.0697, va
l_acc=0.9790
Epoch 36 | lr=0.05000 | train_loss=0.0156, train_acc=0.9981 | val_loss=0.0708, va
l_acc=0.9782
Epoch 37 | lr=0.05000 | train_loss=0.0148, train_acc=0.9985 | val_loss=0.0705, va
l_acc=0.9784
Epoch 38 | lr=0.05000 | train_loss=0.0145, train_acc=0.9987 | val_loss=0.0705, va
l_acc=0.9786
Epoch 39 | lr=0.05000 | train_loss=0.0139, train_acc=0.9987 | val_loss=0.0699, va
l_acc=0.9786
Epoch 40 | lr=0.05000 | train_loss=0.0133, train_acc=0.9987 | val_loss=0.0693, va
l_acc=0.9791
Epoch 41 | lr=0.02500 | train_loss=0.0130, train_acc=0.9988 | val_loss=0.0691, va
l_acc=0.9787
Epoch 42 | lr=0.02500 | train_loss=0.0128, train_acc=0.9988 | val_loss=0.0689, va
l_acc=0.9782
```

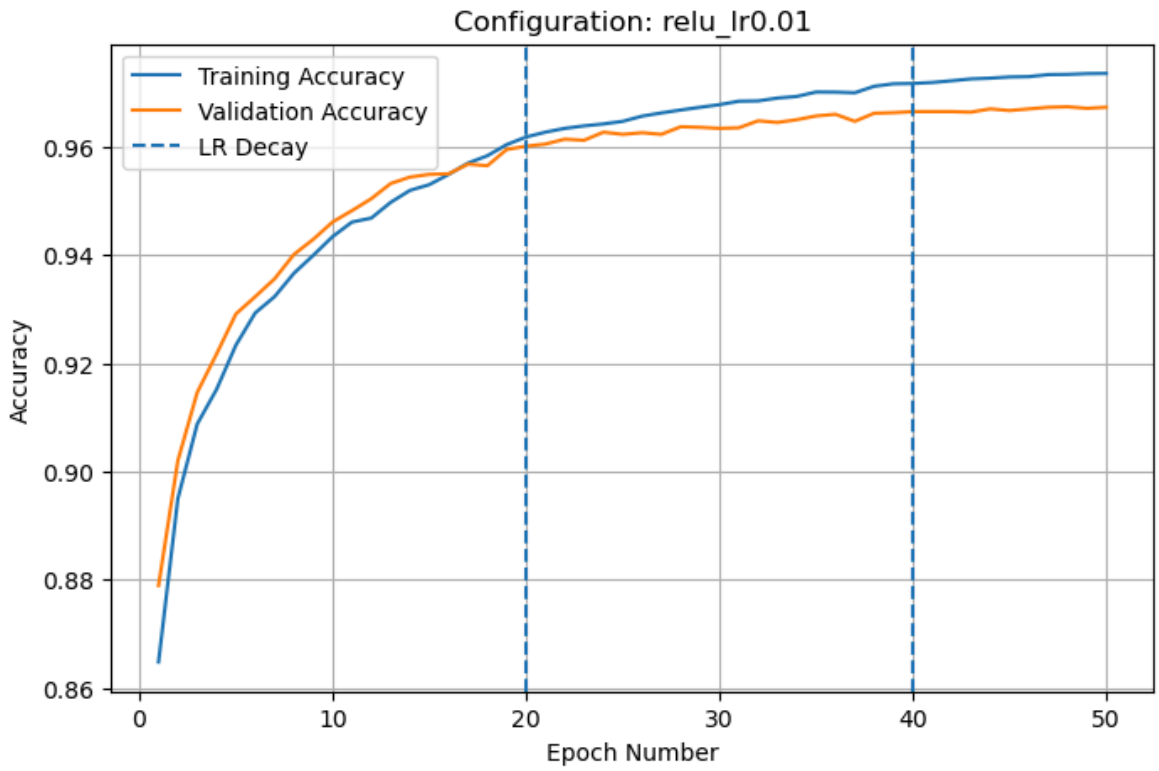
```
Epoch 43 | lr=0.02500 | train_loss=0.0130, train_acc=0.9989 | val_loss=0.0691, va
l_acc=0.9788
Epoch 44 | lr=0.02500 | train_loss=0.0124, train_acc=0.9990 | val_loss=0.0688, va
l_acc=0.9787
Epoch 45 | lr=0.02500 | train_loss=0.0123, train_acc=0.9990 | val_loss=0.0693, va
l_acc=0.9789
Epoch 46 | lr=0.02500 | train_loss=0.0121, train_acc=0.9990 | val_loss=0.0688, va
l_acc=0.9789
Epoch 47 | lr=0.02500 | train_loss=0.0120, train_acc=0.9991 | val_loss=0.0689, va
l_acc=0.9789
Epoch 48 | lr=0.02500 | train_loss=0.0118, train_acc=0.9991 | val_loss=0.0689, va
l_acc=0.9791
Epoch 49 | lr=0.02500 | train_loss=0.0116, train_acc=0.9992 | val_loss=0.0688, va
l_acc=0.9793
Epoch 50 | lr=0.02500 | train_loss=0.0115, train_acc=0.9991 | val_loss=0.0692, va
l_acc=0.9789
Best val acc for tanh_lr0.1: 0.9793

======================================================================
Training config: tanh_lr0.01
======================================================================
Epoch 01 | lr=0.01000 | train_loss=0.5247, train_acc=0.8688 | val_loss=0.4842, va
l_acc=0.8815
Epoch 02 | lr=0.01000 | train_loss=0.4080, train_acc=0.8905 | val_loss=0.3752, va
l_acc=0.9008
Epoch 03 | lr=0.01000 | train_loss=0.3609, train_acc=0.8995 | val_loss=0.3332, va
l_acc=0.9100
Epoch 04 | lr=0.01000 | train_loss=0.3340, train_acc=0.9062 | val_loss=0.3100, va
l_acc=0.9129
Epoch 05 | lr=0.01000 | train_loss=0.3145, train_acc=0.9109 | val_loss=0.2938, va
l_acc=0.9172
Epoch 06 | lr=0.01000 | train_loss=0.3002, train_acc=0.9149 | val_loss=0.2815, va
l_acc=0.9203
Epoch 07 | lr=0.01000 | train_loss=0.2881, train_acc=0.9186 | val_loss=0.2712, va
l_acc=0.9227
Epoch 08 | lr=0.01000 | train_loss=0.2785, train_acc=0.9206 | val_loss=0.2641, va
l_acc=0.9248
Epoch 09 | lr=0.01000 | train_loss=0.2694, train_acc=0.9237 | val_loss=0.2555, va
l_acc=0.9266
Epoch 10 | lr=0.01000 | train_loss=0.2611, train_acc=0.9257 | val_loss=0.2484, va
l_acc=0.9298
Epoch 11 | lr=0.01000 | train_loss=0.2535, train_acc=0.9284 | val_loss=0.2421, va
l_acc=0.9314
Epoch 12 | lr=0.01000 | train_loss=0.2463, train_acc=0.9302 | val_loss=0.2360, va
l_acc=0.9338
Epoch 13 | lr=0.01000 | train_loss=0.2393, train_acc=0.9329 | val_loss=0.2300, va
l_acc=0.9349
Epoch 14 | lr=0.01000 | train_loss=0.2333, train_acc=0.9340 | val_loss=0.2250, va
l_acc=0.9366
Epoch 15 | lr=0.01000 | train_loss=0.2266, train_acc=0.9363 | val_loss=0.2200, va
l_acc=0.9374
Epoch 16 | lr=0.01000 | train_loss=0.2211, train_acc=0.9374 | val_loss=0.2151, va
l_acc=0.9396
Epoch 17 | lr=0.01000 | train_loss=0.2151, train_acc=0.9390 | val_loss=0.2095, va
l_acc=0.9418
Epoch 18 | lr=0.01000 | train_loss=0.2102, train_acc=0.9401 | val_loss=0.2057, va
l_acc=0.9420
Epoch 19 | lr=0.01000 | train_loss=0.2047, train_acc=0.9416 | val_loss=0.2011, va
l_acc=0.9442
Epoch 20 | lr=0.01000 | train_loss=0.1996, train_acc=0.9432 | val_loss=0.1970, va
```

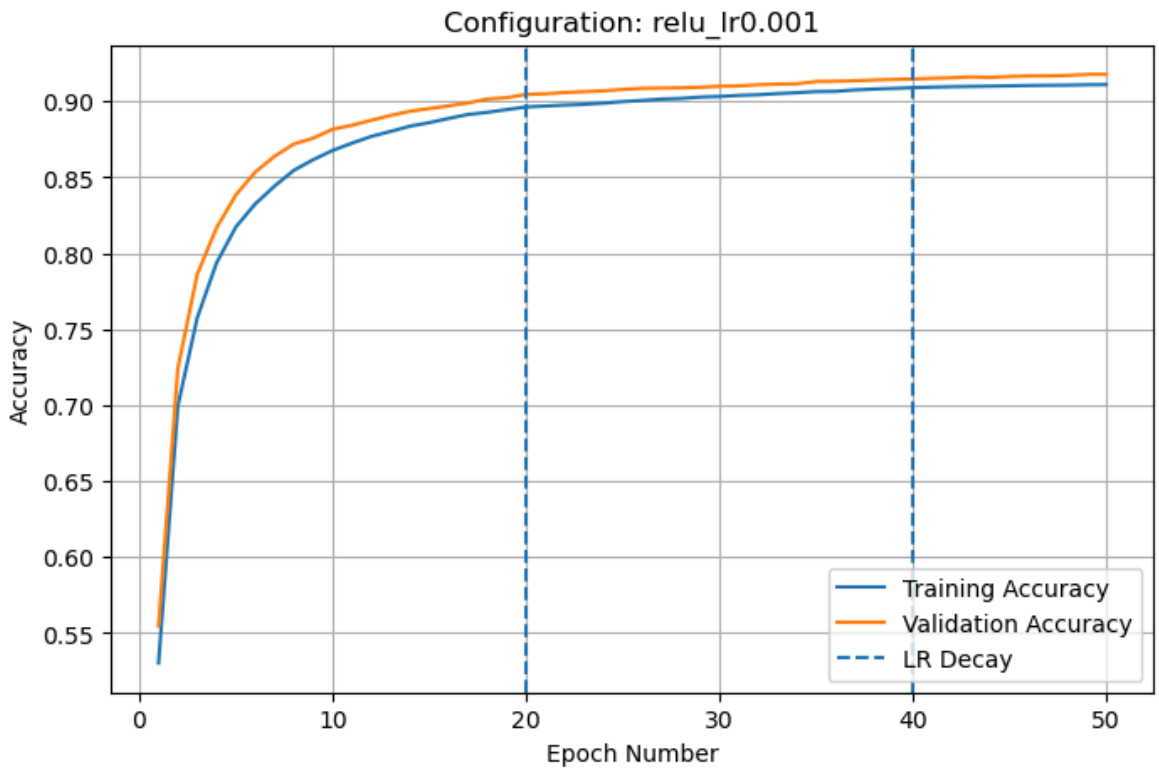
```
l_acc=0.9453
Epoch 21 | lr=0.00500 | train_loss=0.1970, train_acc=0.9439 | val_loss=0.1944, va
l_acc=0.9467
Epoch 22 | lr=0.00500 | train_loss=0.1947, train_acc=0.9445 | val_loss=0.1925, va
l_acc=0.9483
Epoch 23 | lr=0.00500 | train_loss=0.1922, train_acc=0.9451 | val_loss=0.1904, va
l_acc=0.9484
Epoch 24 | lr=0.00500 | train_loss=0.1898, train_acc=0.9459 | val_loss=0.1884, va
l_acc=0.9497
Epoch 25 | lr=0.00500 | train_loss=0.1876, train_acc=0.9465 | val_loss=0.1870, va
l_acc=0.9497
Epoch 26 | lr=0.00500 | train_loss=0.1853, train_acc=0.9472 | val_loss=0.1849, va
l_acc=0.9502
Epoch 27 | lr=0.00500 | train_loss=0.1834, train_acc=0.9478 | val_loss=0.1831, va
l_acc=0.9506
Epoch 28 | lr=0.00500 | train_loss=0.1813, train_acc=0.9484 | val_loss=0.1819, va
l_acc=0.9511
Epoch 29 | lr=0.00500 | train_loss=0.1790, train_acc=0.9491 | val_loss=0.1796, va
l_acc=0.9513
Epoch 30 | lr=0.00500 | train_loss=0.1770, train_acc=0.9497 | val_loss=0.1779, va
l_acc=0.9519
Epoch 31 | lr=0.00500 | train_loss=0.1751, train_acc=0.9498 | val_loss=0.1763, va
l_acc=0.9528
Epoch 32 | lr=0.00500 | train_loss=0.1734, train_acc=0.9505 | val_loss=0.1747, va
l_acc=0.9532
Epoch 33 | lr=0.00500 | train_loss=0.1710, train_acc=0.9511 | val_loss=0.1730, va
l_acc=0.9538
Epoch 34 | lr=0.00500 | train_loss=0.1692, train_acc=0.9517 | val_loss=0.1715, va
l_acc=0.9534
Epoch 35 | lr=0.00500 | train_loss=0.1673, train_acc=0.9522 | val_loss=0.1700, va
l_acc=0.9539
Epoch 36 | lr=0.00500 | train_loss=0.1655, train_acc=0.9528 | val_loss=0.1686, va
l_acc=0.9533
Epoch 37 | lr=0.00500 | train_loss=0.1636, train_acc=0.9531 | val_loss=0.1670, va
l_acc=0.9547
Epoch 38 | lr=0.00500 | train_loss=0.1620, train_acc=0.9535 | val_loss=0.1657, va
l_acc=0.9553
Epoch 39 | lr=0.00500 | train_loss=0.1603, train_acc=0.9540 | val_loss=0.1640, va
l_acc=0.9552
Epoch 40 | lr=0.00500 | train_loss=0.1585, train_acc=0.9545 | val_loss=0.1629, va
l_acc=0.9558
Epoch 41 | lr=0.00250 | train_loss=0.1576, train_acc=0.9548 | val_loss=0.1622, va
l_acc=0.9561
Epoch 42 | lr=0.00250 | train_loss=0.1568, train_acc=0.9552 | val_loss=0.1613, va
l_acc=0.9568
Epoch 43 | lr=0.00250 | train_loss=0.1561, train_acc=0.9553 | val_loss=0.1608, va
l_acc=0.9561
Epoch 44 | lr=0.00250 | train_loss=0.1552, train_acc=0.9555 | val_loss=0.1602, va
l_acc=0.9566
Epoch 45 | lr=0.00250 | train_loss=0.1545, train_acc=0.9560 | val_loss=0.1596, va
l_acc=0.9566
Epoch 46 | lr=0.00250 | train_loss=0.1536, train_acc=0.9562 | val_loss=0.1590, va
l_acc=0.9571
Epoch 47 | lr=0.00250 | train_loss=0.1528, train_acc=0.9564 | val_loss=0.1582, va
l_acc=0.9573
Epoch 48 | lr=0.00250 | train_loss=0.1520, train_acc=0.9564 | val_loss=0.1576, va
l_acc=0.9578
Epoch 49 | lr=0.00250 | train_loss=0.1513, train_acc=0.9567 | val_loss=0.1571, va
l_acc=0.9575
Epoch 50 | lr=0.00250 | train_loss=0.1505, train_acc=0.9567 | val_loss=0.1565, va
```

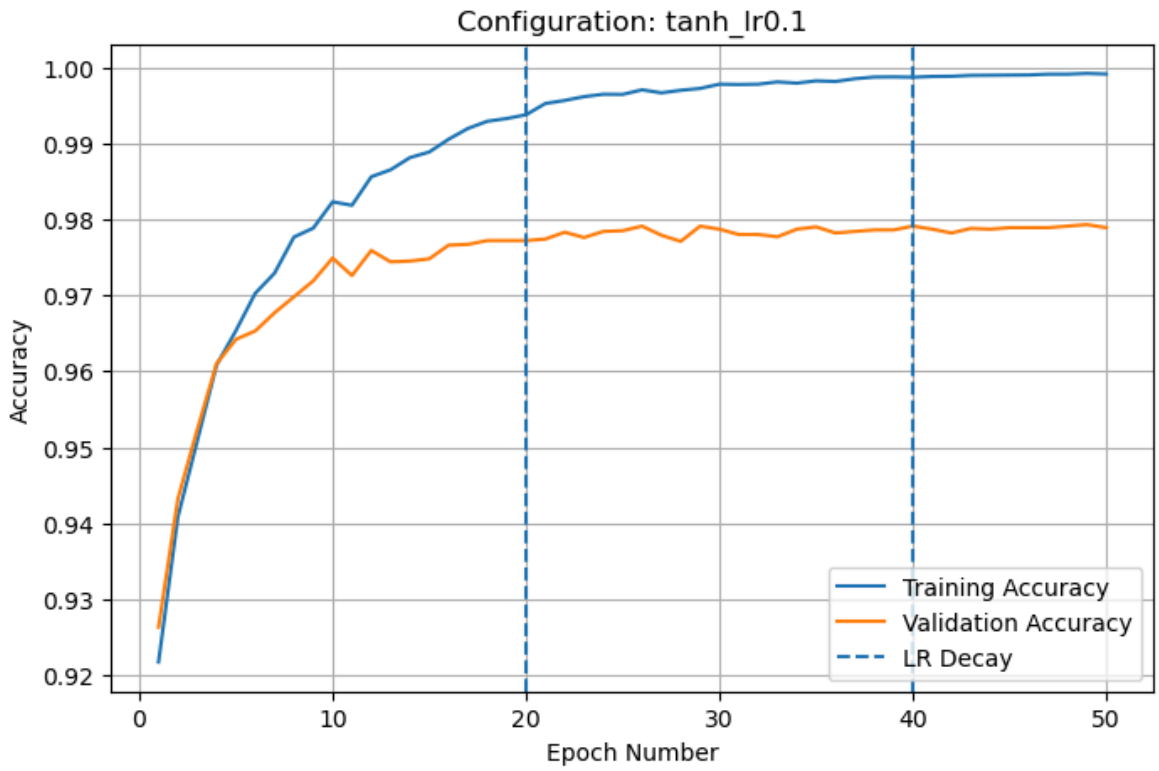
```
l_acc=0.9579
Best val acc for tanh_lr0.01: 0.9579

======================================================================
Training config: tanh_lr0.001
======================================================================
Epoch 01 | lr=0.00100 | train_loss=1.5557, train_acc=0.6492 | val_loss=1.5368, va
l_acc=0.6669
Epoch 02 | lr=0.00100 | train_loss=1.1473, train_acc=0.7580 | val_loss=1.1147, va
l_acc=0.7807
Epoch 03 | lr=0.00100 | train_loss=0.9321, train_acc=0.7992 | val_loss=0.8931, va
l_acc=0.8193
Epoch 04 | lr=0.00100 | train_loss=0.8034, train_acc=0.8213 | val_loss=0.7614, va
l_acc=0.8418
Epoch 05 | lr=0.00100 | train_loss=0.7182, train_acc=0.8351 | val_loss=0.6752, va
l_acc=0.8543
Epoch 06 | lr=0.00100 | train_loss=0.6575, train_acc=0.8465 | val_loss=0.6143, va
l_acc=0.8623
Epoch 07 | lr=0.00100 | train_loss=0.6120, train_acc=0.8546 | val_loss=0.5692, va
l_acc=0.8688
Epoch 08 | lr=0.00100 | train_loss=0.5765, train_acc=0.8599 | val_loss=0.5344, va
l_acc=0.8731
Epoch 09 | lr=0.00100 | train_loss=0.5480, train_acc=0.8651 | val_loss=0.5065, va
l_acc=0.8782
Epoch 10 | lr=0.00100 | train_loss=0.5244, train_acc=0.8688 | val_loss=0.4838, va
l_acc=0.8818
Epoch 11 | lr=0.00100 | train_loss=0.5047, train_acc=0.8731 | val_loss=0.4650, va
l_acc=0.8858
Epoch 12 | lr=0.00100 | train_loss=0.4878, train_acc=0.8759 | val_loss=0.4489, va
l_acc=0.8886
Epoch 13 | lr=0.00100 | train_loss=0.4732, train_acc=0.8779 | val_loss=0.4352, va
l_acc=0.8898
Epoch 14 | lr=0.00100 | train_loss=0.4604, train_acc=0.8803 | val_loss=0.4232, va
l_acc=0.8912
Epoch 15 | lr=0.00100 | train_loss=0.4491, train_acc=0.8826 | val_loss=0.4127, va
l_acc=0.8925
Epoch 16 | lr=0.00100 | train_loss=0.4390, train_acc=0.8844 | val_loss=0.4034, va
l_acc=0.8943
Epoch 17 | lr=0.00100 | train_loss=0.4299, train_acc=0.8860 | val_loss=0.3950, va
l_acc=0.8958
Epoch 18 | lr=0.00100 | train_loss=0.4217, train_acc=0.8874 | val_loss=0.3876, va
l_acc=0.8972
Epoch 19 | lr=0.00100 | train_loss=0.4142, train_acc=0.8889 | val_loss=0.3808, va
l_acc=0.8997
Epoch 20 | lr=0.00100 | train_loss=0.4074, train_acc=0.8905 | val_loss=0.3746, va
l_acc=0.9008
Epoch 21 | lr=0.00050 | train_loss=0.4042, train_acc=0.8911 | val_loss=0.3717, va
l_acc=0.9016
Epoch 22 | lr=0.00050 | train_loss=0.4011, train_acc=0.8918 | val_loss=0.3689, va
l_acc=0.9019
Epoch 23 | lr=0.00050 | train_loss=0.3981, train_acc=0.8923 | val_loss=0.3662, va
l_acc=0.9021
Epoch 24 | lr=0.00050 | train_loss=0.3953, train_acc=0.8929 | val_loss=0.3637, va
l_acc=0.9027
Epoch 25 | lr=0.00050 | train_loss=0.3925, train_acc=0.8934 | val_loss=0.3612, va
l_acc=0.9031
Epoch 26 | lr=0.00050 | train_loss=0.3899, train_acc=0.8941 | val_loss=0.3589, va
l_acc=0.9032
Epoch 27 | lr=0.00050 | train_loss=0.3873, train_acc=0.8946 | val_loss=0.3566, va
l_acc=0.9040
```

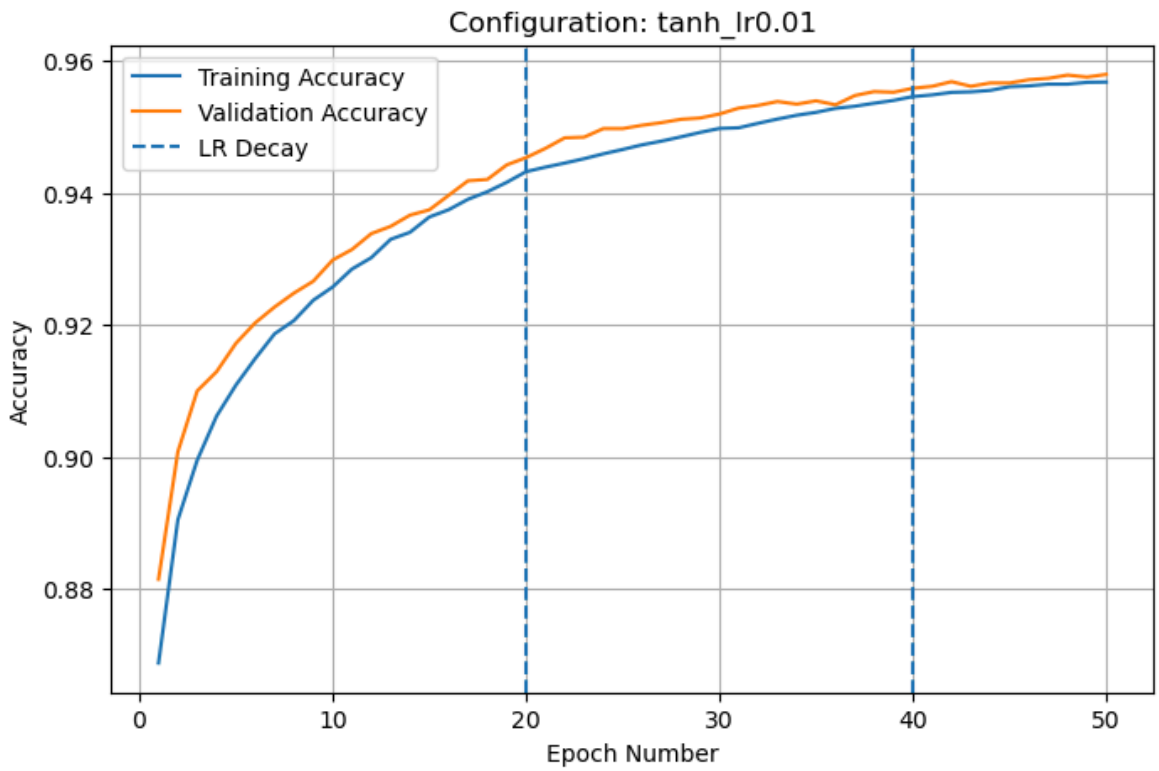
```
Epoch 28 | lr=0.00050 | train_loss=0.3848, train_acc=0.8950 | val_loss=0.3544, va
l_acc=0.9040
Epoch 29 | lr=0.00050 | train_loss=0.3824, train_acc=0.8955 | val_loss=0.3523, va
l_acc=0.9047
Epoch 30 | lr=0.00050 | train_loss=0.3801, train_acc=0.8960 | val_loss=0.3502, va
l_acc=0.9054
Epoch 31 | lr=0.00050 | train_loss=0.3779, train_acc=0.8965 | val_loss=0.3482, va
l_acc=0.9057
Epoch 32 | lr=0.00050 | train_loss=0.3757, train_acc=0.8969 | val_loss=0.3463, va
l_acc=0.9060
Epoch 33 | lr=0.00050 | train_loss=0.3736, train_acc=0.8973 | val_loss=0.3445, va
l_acc=0.9061
Epoch 34 | lr=0.00050 | train_loss=0.3716, train_acc=0.8978 | val_loss=0.3427, va
l_acc=0.9063
Epoch 35 | lr=0.00050 | train_loss=0.3696, train_acc=0.8981 | val_loss=0.3410, va
l_acc=0.9061
Epoch 36 | lr=0.00050 | train_loss=0.3677, train_acc=0.8986 | val_loss=0.3393, va
l_acc=0.9069
Epoch 37 | lr=0.00050 | train_loss=0.3658, train_acc=0.8988 | val_loss=0.3377, va
l_acc=0.9074
Epoch 38 | lr=0.00050 | train_loss=0.3640, train_acc=0.8992 | val_loss=0.3361, va
l_acc=0.9076
Epoch 39 | lr=0.00050 | train_loss=0.3622, train_acc=0.8996 | val_loss=0.3344, va
l_acc=0.9082
Epoch 40 | lr=0.00050 | train_loss=0.3605, train_acc=0.8999 | val_loss=0.3330, va
l_acc=0.9084
Epoch 41 | lr=0.00025 | train_loss=0.3596, train_acc=0.9000 | val_loss=0.3322, va
l_acc=0.9084
Epoch 42 | lr=0.00025 | train_loss=0.3588, train_acc=0.9001 | val_loss=0.3315, va
l_acc=0.9086
Epoch 43 | lr=0.00025 | train_loss=0.3580, train_acc=0.9004 | val_loss=0.3308, va
l_acc=0.9087
Epoch 44 | lr=0.00025 | train_loss=0.3571, train_acc=0.9004 | val_loss=0.3301, va
l_acc=0.9088
Epoch 45 | lr=0.00025 | train_loss=0.3563, train_acc=0.9006 | val_loss=0.3294, va
l_acc=0.9089
Epoch 46 | lr=0.00025 | train_loss=0.3555, train_acc=0.9008 | val_loss=0.3287, va
l_acc=0.9090
Epoch 47 | lr=0.00025 | train_loss=0.3547, train_acc=0.9011 | val_loss=0.3280, va
l_acc=0.9095
Epoch 48 | lr=0.00025 | train_loss=0.3540, train_acc=0.9013 | val_loss=0.3273, va
l_acc=0.9095
Epoch 49 | lr=0.00025 | train_loss=0.3532, train_acc=0.9016 | val_loss=0.3267, va
l_acc=0.9098
Epoch 50 | lr=0.00025 | train_loss=0.3524, train_acc=0.9018 | val_loss=0.3260, va
l_acc=0.9100
Best val acc for tanh_lr0.001: 0.9100

Best configuration:
{'activation': 'relu', 'lr': 0.1, 'name': 'relu_lr0.1'}
Best validation accuracy: 0.9802
```

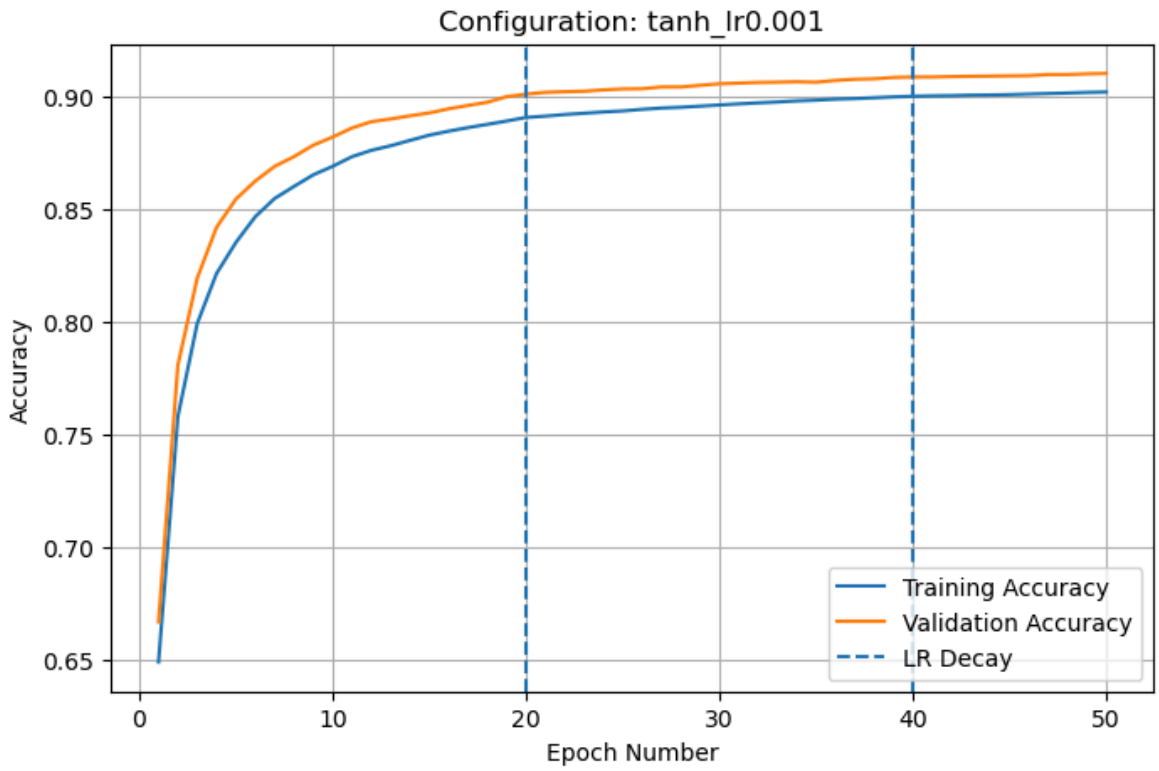

Configuration: relu_lr0.1
Accuracy
Epoch Number
Training Accuracy
Validation Accuracy
LR Decay
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0
10
20
30
40
50


Configuration: relu_lr0.01
Accuracy
Epoch Number
Training Accuracy
Validation Accuracy
LR Decay
0.96
0.94
0.92
0.90
0.88
0.86
0
10
20
30
40
50


Configuration: relu_lr0.001
Accuracy
Epoch Number
Training Accuracy
Validation Accuracy
LR Decay


Configuration: tanh_lr0.1
Accuracy
Epoch Number
Training Accuracy
Validation Accuracy
LR Decay


Configuration: tanh_lr0.01
Training Accuracy
Validation Accuracy
LR Decay
Accuracy
Epoch Number
0.96
0.94
0.92
0.90
0.88
0
10
20
30
40
50


Configuration: tanh_lr0.001
Training Accuracy
Validation Accuracy
LR Decay
Accuracy
Epoch Number
0.90
0.85
0.80
0.75
0.70
0.65
0
10
20
30
40
50

```
======================================================================
Retraining on all 60,000 training images...
======================================================================
[Full Train] Epoch 01 | lr=0.10000 | loss=0.1929, acc=0.9445
[Full Train] Epoch 02 | lr=0.10000 | loss=0.1334, acc=0.9626
[Full Train] Epoch 03 | lr=0.10000 | loss=0.1034, acc=0.9708
[Full Train] Epoch 04 | lr=0.10000 | loss=0.0865, acc=0.9754
[Full Train] Epoch 05 | lr=0.10000 | loss=0.0732, acc=0.9790
[Full Train] Epoch 06 | lr=0.10000 | loss=0.0546, acc=0.9853
[Full Train] Epoch 07 | lr=0.10000 | loss=0.0518, acc=0.9853
[Full Train] Epoch 08 | lr=0.10000 | loss=0.0406, acc=0.9891
[Full Train] Epoch 09 | lr=0.10000 | loss=0.0332, acc=0.9919
[Full Train] Epoch 10 | lr=0.10000 | loss=0.0324, acc=0.9924
[Full Train] Epoch 11 | lr=0.10000 | loss=0.0259, acc=0.9941
[Full Train] Epoch 12 | lr=0.10000 | loss=0.0237, acc=0.9946
[Full Train] Epoch 13 | lr=0.10000 | loss=0.0197, acc=0.9961
[Full Train] Epoch 14 | lr=0.10000 | loss=0.0174, acc=0.9970
[Full Train] Epoch 15 | lr=0.10000 | loss=0.0169, acc=0.9970
[Full Train] Epoch 16 | lr=0.10000 | loss=0.0133, acc=0.9979
[Full Train] Epoch 17 | lr=0.10000 | loss=0.0135, acc=0.9981
[Full Train] Epoch 18 | lr=0.10000 | loss=0.0134, acc=0.9977
[Full Train] Epoch 19 | lr=0.10000 | loss=0.0107, acc=0.9988
[Full Train] Epoch 20 | lr=0.10000 | loss=0.0085, acc=0.9993
[Full Train] Epoch 21 | lr=0.05000 | loss=0.0074, acc=0.9995
[Full Train] Epoch 22 | lr=0.05000 | loss=0.0070, acc=0.9996
[Full Train] Epoch 23 | lr=0.05000 | loss=0.0067, acc=0.9996
[Full Train] Epoch 24 | lr=0.05000 | loss=0.0066, acc=0.9996
[Full Train] Epoch 25 | lr=0.05000 | loss=0.0063, acc=0.9997
[Full Train] Epoch 26 | lr=0.05000 | loss=0.0059, acc=0.9997
[Full Train] Epoch 27 | lr=0.05000 | loss=0.0059, acc=0.9997
[Full Train] Epoch 28 | lr=0.05000 | loss=0.0057, acc=0.9998
[Full Train] Epoch 29 | lr=0.05000 | loss=0.0055, acc=0.9998
[Full Train] Epoch 30 | lr=0.05000 | loss=0.0052, acc=0.9998
[Full Train] Epoch 31 | lr=0.05000 | loss=0.0050, acc=0.9999
[Full Train] Epoch 32 | lr=0.05000 | loss=0.0051, acc=0.9998
[Full Train] Epoch 33 | lr=0.05000 | loss=0.0048, acc=0.9999
[Full Train] Epoch 34 | lr=0.05000 | loss=0.0048, acc=0.9999
[Full Train] Epoch 35 | lr=0.05000 | loss=0.0046, acc=0.9999
[Full Train] Epoch 36 | lr=0.05000 | loss=0.0045, acc=0.9999
[Full Train] Epoch 37 | lr=0.05000 | loss=0.0045, acc=1.0000
[Full Train] Epoch 38 | lr=0.05000 | loss=0.0043, acc=0.9999
[Full Train] Epoch 39 | lr=0.05000 | loss=0.0043, acc=0.9999
[Full Train] Epoch 40 | lr=0.05000 | loss=0.0042, acc=1.0000
[Full Train] Epoch 41 | lr=0.02500 | loss=0.0040, acc=1.0000
[Full Train] Epoch 42 | lr=0.02500 | loss=0.0040, acc=1.0000
[Full Train] Epoch 43 | lr=0.02500 | loss=0.0039, acc=1.0000
[Full Train] Epoch 44 | lr=0.02500 | loss=0.0039, acc=1.0000
[Full Train] Epoch 45 | lr=0.02500 | loss=0.0039, acc=1.0000
[Full Train] Epoch 46 | lr=0.02500 | loss=0.0038, acc=1.0000
[Full Train] Epoch 47 | lr=0.02500 | loss=0.0038, acc=1.0000
[Full Train] Epoch 48 | lr=0.02500 | loss=0.0038, acc=1.0000
[Full Train] Epoch 49 | lr=0.02500 | loss=0.0037, acc=1.0000
[Full Train] Epoch 50 | lr=0.02500 | loss=0.0037, acc=1.0000

Final Test Accuracy: 0.9797
Final Test Loss: 0.06327622

===== Summary =====
Best config: {'activation': 'relu', 'lr': 0.1, 'name': 'relu_lr0.1'}
Best val acc: 0.9802
```

```
Final test acc: 0.9797
Final test loss: 0.06327622
```

Network Configuration

I used a fully connected multilayer perceptron with architecture 784-256-128-10. The input layer has 784 neurons, corresponding to the 28×28 MNIST image pixels. There are two hidden layers with 256 and 128 neurons respectively. The output layer has 10 neurons, one for each digit class. ReLU and tanh were tested as hidden-layer activation functions. The output layer uses softmax.

Batch

I used a minibatch size of 100. This divides the 50,000 training images evenly into 500 updates per epoch.

Initial Learning Rates

I tested three initial learning rates: 0.1, 0.01, and 0.001. For each learning rate, I trained the model using both ReLU and tanh activation functions, resulting in 6 configurations in total. The learning rate was reduced by a factor of 2 after epoch 20 and epoch 40.

Parameter Initialization

I did not initialize weights with zeros. For hidden layers using ReLU, I used He initialization. For hidden layers using tanh, I used Xavier initialization. The output layer was initialized using Xavier initialization. All bias vectors were initialized to zero.

Training and Validation Accuracy Curves

I trained 6 configurations in total:

1. ReLU, learning rate = 0.1
2. ReLU, learning rate = 0.01
3. ReLU, learning rate = 0.001
4. tanh, learning rate = 0.1
5. tanh, learning rate = 0.01
6. tanh, learning rate = 0.001

For each configuration, I plotted both training accuracy and validation accuracy versus epoch number. The epochs where learning rate decay was applied (epoch 20 and epoch 40) were marked on each plot.

From the plots, configurations with learning rate 0.1 converged faster and achieved higher validation accuracy. The models with learning rate 0.001 improved much more slowly and had lower final accuracy. Among all six configurations, ReLU with initial learning rate 0.1 achieved the best validation performance.

Best Model and Final Test Accuracy

The best configuration was ReLU with initial learning rate 0.1. Its best validation accuracy was 0.9802 (98.02%). After selecting this configuration, I retrained the network on all 60,000 training images. I then evaluated the final model on the test set. The final test accuracy was 0.9797 (97.97%).

In [ ]: